MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 21 — Concurrence et Parallélisme

Threads, processus, asyncio — faire plusieurs choses à la fois

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

19 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

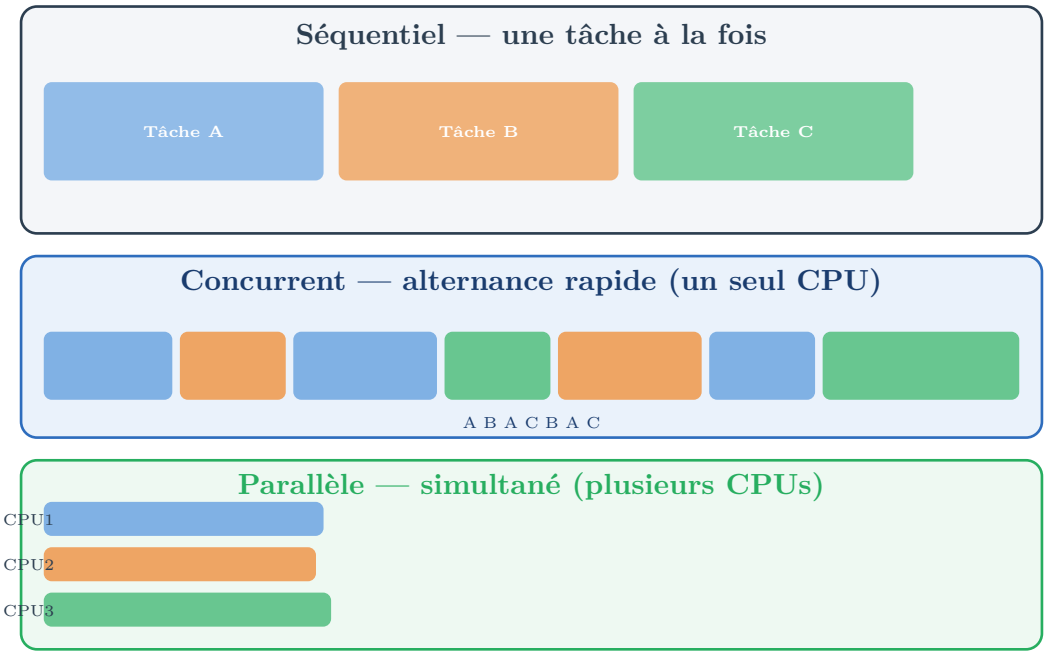
Chapitre 21 — Concurrence et Parallélisme **2**
0.1 Concurrence vs Parallélisme — Les concepts 2
0.2 threading — Exécution concurrente 2
0.3 Synchronisation — Lock et Event 3
0.4 ThreadPoolExecutor — Pool de threads 5
0.5 multiprocessing — Vrai parallélisme CPU 6
0.6 asyncio — Programmation asynchrone 7
0.7 Choisir la bonne approche 9
0.8 Ce qui m'a le plus surpris dans ce chapitre 9
0.9 Résumé du Chapitre 21 10

Chapitre 21 — Concurrence et Parallélisme

À retenir

La **concurrence** et le **parallélisme** permettent à un programme d'exécuter plusieurs tâches simultanément. Python offre trois approches : `threading` pour les tâches I/O-bound, `multiprocessing` pour les tâches CPU-bound, et `asyncio` pour la programmation asynchrone. Choisir la bonne approche selon le contexte est l'une des compétences clés du développeur Python avancé.

Concurrence vs Parallélisme — Les concepts



À retenir

Le GIL (Global Interpreter Lock) est un verrou Python qui empêche plusieurs threads d'exécuter du bytecode Python simultanément. Conséquence : le threading Python ne donne pas de vrai parallélisme pour les tâches CPU. Solution : `multiprocessing` pour le CPU, `threading`/`asyncio` pour l'I/O.

threading — Exécution concurrente

threading.Thread — créer des threads

```
import threading
import time

```

```python
def telecharger(url, duree):
    """Simule un telechargement."""
    print(f"  [{threading.current_thread().name}] "
          f"Debut : {url}")
    time.sleep(duree)    # simule l'attente reseau
    print(f"  [{threading.current_thread().name}] "
          f"Fin   : {url} ({duree}s)")

# SEQUENTIEL     6 secondes au total
print("=== Sequentiel ===")
debut = time.time()
telecharger("page1.html", 2)
telecharger("page2.html", 1)
telecharger("page3.html", 3)
print(f"Dur e : {time.time()-debut:.1f}s\n")

# CONCURRENT avec threads     ~3 secondes
print("=== Concurrent (threads) ===")
debut = time.time()

threads = [
    threading.Thread(target=telecharger,
                     args=("page1.html", 2),
                     name="T1"),
    threading.Thread(target=telecharger,
                     args=("page2.html", 1),
                     name="T2"),
    threading.Thread(target=telecharger,
                     args=("page3.html", 3),
                     name="T3"),
]

for t in threads:
    t.start()       # demarrer chaque thread

for t in threads:
    t.join()        # attendre la fin de chaque thread

print(f"Duree : {time.time()-debut:.1f}s")
#     ~3s au lieu de 6s !
```

Synchronisation — Lock et Event

Lock — protéger les ressources partagées

```python
import threading

# PROBLEME sans Lock     race condition
```

```python
compteur = 0

def incrementer_sans_lock(n):
    global compteur
    for _ in range(n):
        compteur += 1    # NON atomique     danger !

# SOLUTION avec Lock
compteur_safe = 0
lock = threading.Lock()

def incrementer_avec_lock(n):
    global compteur_safe
    for _ in range(n):
        with lock:           # acquiert et relache
           automatiquement
            compteur_safe += 1

# Demonstration
threads = [threading.Thread(target=incrementer_avec_lock,
                            args=(10000,))
           for _ in range(10)]

for t in threads: t.start()
for t in threads: t.join()

print(f"Resultat attendu : 100000")
print(f"Resultat obtenu  : {compteur_safe}")
#     100000 (correct avec lock)

# Event     synchroniser des threads
pret = threading.Event()

def producteur():
    print("Producteur : preparation...")
    time.sleep(2)
    print("Producteur : donnees pretes !")
    pret.set()    # signaler que c'est pret

def consommateur():
    print("Consommateur : en attente...")
    pret.wait()   # bloque jusqu'au signal
    print("Consommateur : traitement des donnees !")

t1 = threading.Thread(target=producteur)
t2 = threading.Thread(target=consommateur)
t1.start(); t2.start()
t1.join();  t2.join()
```

ThreadPoolExecutor — Pool de threads

concurrent.futures — pool de threads élégant

```python
from concurrent.futures import ThreadPoolExecutor, as_completed
import time

def analyser_url(url):
    """Simule l'analyse d'une URL."""
    time.sleep(1)   # simule une requete reseau
    return f"Resultat pour {url} : OK"

urls = [f"https://site{i}.com" for i in range(1, 9)]

# Sans pool     8 secondes
# Avec pool de 4 threads     ~2 secondes
debut = time.time()

with ThreadPoolExecutor(max_workers=4) as executor:
    # map()     soumettre toutes les taches et collecter
    resultats = list(executor.map(analyser_url, urls))

print(f"Duree : {time.time()-debut:.1f}s")
for r in resultats:
    print(f"  {r}")

# as_completed()     traiter les resultats au fur et a mesure
debut = time.time()

with ThreadPoolExecutor(max_workers=4) as executor:
    # submit()     soumettre individuellement
    futures = {executor.submit(analyser_url, url): url
               for url in urls}

    for future in as_completed(futures):
        url = futures[future]
        try:
            resultat = future.result()
            print(f"      {resultat}")
        except Exception as e:
            print(f"      {url} : {e}")

print(f"Duree : {time.time()-debut:.1f}s")
```

multiprocessing — Vrai parallélisme CPU

multiprocessing — contourner le GIL

```python
import multiprocessing
import time

def calcul_intensif(n):
    """T che CPU-bound     calcul lourd."""
    total = 0
    for i in range(n):
        total += i ** 2
    return total

if __name__ == "__main__":

    N = 5_000_000

    # SEQUENTIEL
    debut = time.time()
    resultats = [calcul_intensif(N) for _ in range(4)]
    print(f"Sequentiel : {time.time()-debut:.2f}s")

    # PARALLELE avec Pool
    debut = time.time()
    with multiprocessing.Pool(processes=4) as pool:
        resultats = pool.map(calcul_intensif, [N]*4)
    print(f"Parallele  : {time.time()-debut:.2f}s")
    #     ~4x plus rapide sur 4 c urs !

    # Process individuel
    def worker(nom, queue):
        resultat = calcul_intensif(1_000_000)
        queue.put((nom, resultat))

    queue = multiprocessing.Queue()
    processes = [
        multiprocessing.Process(target=worker,
                                args=(f"P{i}", queue))
        for i in range(4)
    ]

    for p in processes: p.start()
    for p in processes: p.join()

    while not queue.empty():
        nom, res = queue.get()
        print(f"  {nom} : {res:,}")
```

Note importante

Le bloc `if __name__ == "__main__"` est **obligatoire** avec `multiprocessing` sur Windows. Sans lui, chaque processus enfant réimporte le module et relance la création de processus — boucle infinie. Sur Linux/Mac ce n'est pas requis, mais c'est une bonne pratique universelle.

asyncio — Programmation asynchrone

async / await — coroutines asynchrones

```python
import asyncio
import time

# Fonction asynchrone      coroutine
async def telecharger_async(url, duree):
    print(f"  Debut : {url}")
    await asyncio.sleep(duree)    # attend SANS bloquer la
        boucle
    print(f"  Fin   : {url} ({duree}s)")
    return f"Contenu de {url}"

# Lancer une seule coroutine
async def main_simple():
    resultat = await telecharger_async("page.html", 1)
    print(resultat)

asyncio.run(main_simple())

# Lancer plusieurs coroutines EN PARALLELE
async def main_concurrent():
    debut = time.time()

    # gather()      lancer toutes les coroutines simultanement
    resultats = await asyncio.gather(
        telecharger_async("page1.html", 2),
        telecharger_async("page2.html", 1),
        telecharger_async("page3.html", 3),
    )

    print(f"\n Duree totale : {time.time()-debut:.1f}s")
    for r in resultats:
        print(f"      {r}")

asyncio.run(main_concurrent())
#      ~3s au lieu de 6s, sans thread, sans processus !
```

asyncio — patterns avancés

```python
import asyncio

# Timeout      annuler si trop long
async def tache_longue():
    await asyncio.sleep(10)
    return "Termine"

async def avec_timeout():
    try:
        resultat = await asyncio.wait_for(
            tache_longue(), timeout=3.0
        )
    except asyncio.TimeoutError:
        print("Timeout ! Tache annulee apres 3s.")

asyncio.run(avec_timeout())

# Semaphore      limiter la concurrence
async def requete(session_id, semaphore):
    async with semaphore:    # max N requetes simultanees
        print(f"Requete {session_id} en cours...")
        await asyncio.sleep(1)
        print(f"Requ te {session_id} terminee.")

async def main_semaphore():
    # Seulement 3 requetes simultanees max
    sem = asyncio.Semaphore(3)
    taches = [requete(i, sem) for i in range(8)]
    await asyncio.gather(*taches)

asyncio.run(main_semaphore())

# Queue asynchrone      producteur / consommateur
async def producteur_async(queue):
    for i in range(5):
        await queue.put(f"item_{i}")
        await asyncio.sleep(0.5)
    await queue.put(None)    # signal de fin

async def consommateur_async(queue):
    while True:
        item = await queue.get()
        if item is None:
            break
        print(f"Traitement : {item}")
        await asyncio.sleep(0.2)

async def main_queue():
```

```
    queue = asyncio.Queue()
    await asyncio.gather(
        producteur_async(queue),
        consommateur_async(queue)
    )

asyncio.run(main_queue())
```

Choisir la bonne approche

Approche	Cas d'usage	Avantages	Limites
`threading`	I/O-bound : réseau, fichiers, BDD	Simple, mémoire partagée	GIL, race conditions
`multiproc.`	CPU-bound : calcul, ML, image	Vrai parallélisme, pas de GIL	Lourd, mémoire séparée
`asyncio`	I/O massif : APIs, web, BDD async	Très léger, scalable, élégant	async partout, courbe d'apprentissage

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. Le GIL — le grand malentendu sur les threads Python. Je pensais que les threads Python permettaient un vrai parallélisme. Le GIL m'a surpris : en Python, un seul thread exécute du bytecode à la fois. Les threads restent utiles pour l'I/O — pendant qu'un thread attend une réponse réseau, un autre peut tourner. Mais pour du calcul intensif, les threads Python ne donnent aucun gain. C'est une subtilité architecturale majeure, souvent mal comprise même par des développeurs expérimentés.

2. `await` ne bloque pas — il *cède* le contrôle. La subtilité d'`asyncio` est que `await` ne met pas le programme en pause — il dit à la boucle d'événements "je n'ai rien à faire maintenant, exécute une autre coroutine". C'est de la coopération volontaire, pas de la préemption. Une coroutine qui ne fait jamais `await` monopolise la boucle entière. Ce modèle mental — céder plutôt que bloquer — est un vrai changement de paradigme.

3. `ThreadPoolExecutor` simplifie tout. Créer des threads manuellement avec `Thread()`, gérer `start()` et `join()`, collecter les résultats — c'est verbeux. `ThreadPoolExecutor` avec `map()` ou `submit()` réduit tout ça à quelques lignes propres. C'est l'exemple parfait de Python qui fournit une abstraction de haut niveau pour cacher la complexité de bas niveau.

Résumé du Chapitre 21

Ce que j'ai appris dans ce chapitre :

- **GIL** — un seul thread Python actif à la fois pour le bytecode.
- `threading.Thread` — concurrence pour les tâches I/O-bound.
- `Lock` et `Event` — synchroniser les threads, éviter les race conditions.
- `ThreadPoolExecutor` — pool de threads élégant avec `map()` et `submit()`.
- `multiprocessing.Pool` — vrai parallélisme pour les tâches CPU-bound.
- `async def` / `await` — définir et appeler des coroutines.
- `asyncio.gather()` — exécuter plusieurs coroutines simultanément.
- `asyncio.Semaphore` et `asyncio.Queue` — contrôle de flux asynchrone.

"La concurrence, c'est gérer plusieurs choses à la fois. Le parallélisme, c'est les faire vraiment en même temps. Python offre les deux — à condition de choisir le bon outil."